Circle STARK 핵심 개념 및 구현 분석

개요

Circle STARK는 Stwo 프로젝트의 핵심 혁신으로, 기존의 FFT 기반 STARK 시스템과 달리 복소수 원(Complex Circle) 위에서 동작하는 증명 시스템입니다. 이 문서는 Circle STARK의 핵심 개념인 Circle 기하학과 Circle FFT의 구현 방식을 상세히 분석합니다.

1. Circle 기하학과 복소수 원 구현 분석

1.1 CirclePoint 구조체의 수학적 기반

Circle STARK에서 사용하는 복소수 원은 단위원 $x^2 + y^2 = 1$ 을 기반으로 합니다. `CirclePoint<F>` 구조체는 이 원 위의 점을 나타내며, 가법군(additive group)으로 취급됩니다.

```
#[derive(Copy, Clone, Debug, Default, PartialEq, Eq, Hash)]
pub struct CirclePoint<F> {
pub x: F,
pub y: F,
}
```

수학적 배경

- **단위원 조건**: 모든 유효한 CirclePoint는 $x^2 + y^2 = 1$ 을 만족합니다
- **가법군 구조**: 원 위의 점들은 복소수 곱셈을 통해 "덧셈" 연산을 정의합니다
- **항등원**: `(1, 0)` 이 가법군의 항등원(zero element)입니다

1.2 복소수 원 위의 점 연산

Circle 위의 점 연산은 복소수 곱셈 규칙을 따릅니다:

```
impl<F> Add for CirclePoint<F> {
type Output = Self;
fn add(self, rhs: Self) -> Self::Output {
let x = self.x.clone() * rhs.x.clone() - self.y.clone() * rhs.y.clone();
let y = self.x * rhs.y + self.y * rhs.x;
Self { x, y }
}
}
```

이는 복소수 `(a + bi) * (c + di) = (ac - bd) + (ad + bc)i` 와 동일한 연산입니다.

주요 연산들

1. 점 배가 (Point Doubling)

```
pub fn double(&self) -> Self {
self.clone() + self.clone()
}
```

2. x-좌표 배가 최적화

```
pub fn double_x(x: F) -> F {
let sx = x.square();
```

```
sx.clone() + sx - F::one() // 2x² - 1
}
```

이 최적화는 `x` 좌표만으로 배가된 점의 `x` 좌표를 계산할 수 있다는 수학적 성질을 활용합니다.

3. 스칼라 곱셈 (Scalar Multiplication)

```
pub fn mul(&self, mut scalar: u128) -> CirclePoint<F> {
let mut res = Self::zero();
let mut cur = self.clone();
while scalar > 0 {
if scalar & 1 == 1 {
res = res + cur.clone();
}
cur = cur.double();
scalar >>= 1;
}
res
}
```

이는 이진 지수법(binary exponentiation)을 사용한 효율적인 스칼라 곱셈입니다.

1.3 Circle 생성원과 부분군 구조

M31 필드에서의 Circle 생성원

```
pub const M31_CIRCLE_GEN: CirclePoint<M31> = CirclePoint {
```

```
x: M31::from_u32_unchecked(2),
y: M31::from_u32_unchecked(1268011823),
};
pub const M31_CIRCLE_LOG_ORDER: u32 = 31;
```

이 생성원은 `2^31` 차수를 가지며, M31 필드에서 가능한 최대 부분군을 생성합니다.

부분군 생성과 Coset 구조

```
impl CirclePointIndex {
pub fn subgroup_gen(log_size: u32) -> Self {
assert!(log_size <= M31_CIRCLE_LOG_ORDER);
Self(1 << (M31_CIRCLE_LOG_ORDER - log_size))
}
}
```

부분군 생성원은 `G_{2^k} = 2^{31-k} * G` 로 계산되며, 이는 `2^k` 크기의 부분군을 생성합니다.

Coset 구조와 도메인 구성

```
pub struct Coset {
pub initial_index: CirclePointIndex,
pub initial: CirclePoint<M31>,
pub step_size: CirclePointIndex,
pub step: CirclePoint<M31>,
```

```
pub log_size: u32,
}
```

Coset은 `initial + <step>` 형태로 정의되며, 다음과 같은 특별한 구조들을 제공합니다:

- **부분군**: `<G_n>` - 크기 `2^n` 의 부분군
- **홀수 Coset**: `G_{2n} + <G_n>` - 홀수 인덱스들
- **반-홀수 Coset**: `G_{4n} + <G_n>` - 특별한 대칭성을 가진 구조

1.4 Circle 도메인의 수학적 특성

Circle 도메인은 두 개의 켤레 coset의 분리합집합으로 구성됩니다:

```
pub struct CircleDomain {
pub half_coset: Coset,
}
```

도메인은 `±C + <G_n>` 형태로, 첫 번째 절반은 `C + <G_n>` , 두 번째 절반은 `-C - <G_n>` 입니다.

정준 도메인 (Canonic Domain)

```
pub fn is_canonic(&self) -> bool {
self.half_coset.initial_index * 4 == self.half_coset.step_size
}
```

정준 도메인은 `G_{2n} + <G_n>` 형태로, 특별한 대칭성과 효율적인 FFT 연산을 가능하게 합니다.

2. Circle FFT 구현 및 최적화 기법 분석

2.1 Circle FFT의 수학적 원리

Circle FFT는 기존 FFT와 달리 복소수 원 위에서 동작하는 고속 변환입니다. 핵심 아이디어는 다음과 같습니다:

1. **Circle 도메인**: 단위원 위의 점들을 평가 도메인으로 사용
2. **이중 구조**: 각 도메인이 coset과 그 켤레의 합집합
3. **계층적 분해**: 재귀적인 배가 연산을 통한 분할정복
4. **Folding 구조**: 계층적 매핑을 통한 효율적인 점별 평가

수학적 기반

Circle FFT는 다음 수학적 구조를 기반으로 합니다:

- **다항식 기저**: FFT 기저는 `{y, x, π(x), π²(x), ..., π^{log_size-2}(x)}` 의 텐서곱
- **배가 매핑**: `π(x) := 2x² - 1` 을 통한 x-좌표 배가
- **Folding 연산**: `fold(values, [x, y, z]) = values[0] + values[1]*x + values[2]*y + values[3]*z + ...`

2.2 Circle FFT 구현 구조

기본 Butterfly 연산

Circle FFT의 핵심은 두 가지 butterfly 연산입니다:

```
// 순방향 butterfly (FFT)
pub fn butterfly<F>(v0: &mut F, v1: &mut F, twid: BaseField) {
let tmp = *v1 * twid;
*v1 = *v0 - tmp;
*v0 += tmp;
}
```

```
// 역방향 butterfly (IFFT)
pub fn ibutterfly<F>(v0: &mut F, v1: &mut F, itwid: BaseField) {
let tmp = *v0;
*v0 = tmp + *v1;
*v1 = (tmp - *v1) * itwid;
}
```

계층별 처리 구조

Circle FFT는 두 종류의 계층으로 구성됩니다:

1. **Circle 계층 (Layer 0)**: Circle 특유의 기하학적 변환
2. **Line 계층 (Layer 1+)**: 전통적인 FFT와 유사한 선형 변환

```
// 평가 (FFT) 과정
for (layer, layer_twiddles) in line_twiddles.iter().enumerate().rev() {
for (h, &t) in layer_twiddles.iter().enumerate() {
fft_layer_loop(&mut values, layer + 1, h, t, butterfly);
}
}
for (h, t) in circle_twiddles.enumerate() {
fft_layer_loop(&mut values, 0, h, t, butterfly);
}
```

```
// 보간 (IFFT) 과정 (역순)
for (h, t) in circle_twiddles.enumerate() {
fft_layer_loop(&mut values, 0, h, t, ibutterfly);
}
for (layer, layer_twiddles) in line_twiddles.into_iter().enumerate() {
for (h, &t) in layer_twiddles.iter().enumerate() {
fft_layer_loop(&mut values, layer + 1, h, t, ibutterfly);
}
}
```

점별 평가 최적화

단일 점에서의 다항식 평가는 folding 기법을 사용합니다:

```
pub fn eval_at_point(poly: &CirclePoly<Self>, point: CirclePoint<SecureField>)
let mut mappings = vec![point.y];
let mut x = point.x;
for _ in 1..poly.log_size() {
mappings.push(x);
x = CirclePoint::double_x(x); // π(x) = 2x² - 1
}
mappings.reverse();
fold(&poly.coeffs, &mappings)
}
```

2.3 Twiddle Factor 계산 최적화

Circle Twiddle 생성

Circle 계층의 twiddle factor는 line twiddle로부터 효율적으로 계산됩니다:

```
fn  circle_twiddles_from_line_twiddles(
first_line_twiddles: &[BaseField],
) -> impl Iterator<Item = BaseField> + '_ {
// Circle coset의 4-fold 대칭성 활용:
// [(x, y), (-x, -y), (y, -x), (-y, x)]
// twiddles: [y, -y, -x, x]
first_line_twiddles
.iter()
.array_chunks()
.flat_map(|[&x, &y]| [y, -y, -x, x])
}
```

이는 Circle의 4-fold 대칭성을 활용한 최적화로, M31_CIRCLE_GEN의 특성을 이용합니다.

배치 역원 계산

대규모 twiddle factor의 역원을 효율적으로 계산하기 위해 배치 처리를 사용합니다:

```
const CHUNK_SIZE: usize = 1 << 12; // 4096
```

```
twiddles
```

```
.array_chunks::<CHUNK_SIZE>()
.zip(itwiddles.array_chunks_mut::<CHUNK_SIZE>())
.for_each(|(src, dst)| {
batch_inverse_in_place(src, dst);
});
```

2.4 SIMD 및 하드웨어 최적화

SIMD 벡터화

SIMD 백엔드는 AVX512를 활용하여 16개 원소를 동시에 처리합니다:

```
const N_LANES: usize = 16; // AVX512 레인 수
const LOG_N_LANES: u32 = 4;

// PackedBaseField를 사용한 벡터화된 연산
type PackedBaseField = PackedM31;
```

메모리 접근 패턴 최적화

1. **Bit-reverse 순서**: 캐시 효율성을 위한 메모리 접근 패턴
2. **청크 단위 처리**: 4KB 청크로 나누어 캐시 미스 최소화
3. **전치 최적화**: 대규모 FFT에서 메모리 접근 패턴 개선

```
// 대규모 FFT에서의 전치 최적화
if  log_size > CACHED_FFT_LOG_SIZE {
mappings.reverse();
```

```
let n = mappings.len();
let n0 = (n - LOG_N_LANES as usize) / 2;
let n1 = (n - LOG_N_LANES as usize).div_ceil(2);
let (ab, c) = mappings.split_at_mut(n1);
let (a, _b) = ab.split_at_mut(n0);
a.swap_with_slice(&mut c[0..n0]);
mappings.reverse();
}
```

병렬 처리 최적화

Rayon을 활용한 병렬 처리로 멀티코어 성능을 활용합니다:

```
#[cfg(feature = "parallel")]
let sum: PackedSecureField = {
const CHUNK_SIZE: usize = 1 << 10;
let chunks = poly.coeffs.data.par_chunks(CHUNK_SIZE).enumerate();
chunks
.into_par_iter()
.map(|(i, chunk)| compute_chunk_sum(chunk, twiddle_mids, i * CHUNK_SIZE))
.sum()
};
```

2.5 기존 FFT와의 차이점 및 장점

주요 차이점

1. **도메인 구조**:

- 기존 FFT: 단위근의 거듭제곱 $\{1, \omega, \omega^2, ..., \omega^{n-1}\}$
- Circle FFT: 복소수 원 위의 점들 $\{(x_i, y_i) \mid x_i^2 + y_i^2 = 1\}$

2. **대칭성**:

- 기존 FFT: 회전 대칭성만 활용
- Circle FFT: 켤레 대칭성 + 회전 대칭성 + 4-fold 대칭성

3. **연산 복잡도**:

- 둘 다 O(n log n)이지만 Circle FFT는 더 적은 필드 곱셈 필요

4. **보간/평가 구조**:

- 기존 FFT: 단일 계층 구조
- Circle FFT: Circle + Line 이중 계층 구조

Circle FFT의 장점

1. **보안성 향상**:

- 더 복잡한 대수적 구조로 인한 공격 저항성 증가
- 켤레 대칭성을 통한 추가적인 제약 조건
- Circle 기하학의 특성으로 인한 구조적 견고성

2. **효율성**:

- Circle 기하학의 특성을 활용한 최적화 가능
- 4-fold 대칭성으로 twiddle factor 계산 비용 절약
- 특정 연산에서 더 적은 필드 연산 필요

3. **유연성**:

- 다양한 크기의 도메인 지원
- 계층적 구조를 통한 적응적 처리
- 부분 도메인 평가 최적화

2.6 성능 최적화 기법

작은 크기 도메인 특별 처리

크기 1, 2, 4인 도메인에 대해서는 특별히 최적화된 구현을 제공합니다:

```
// 크기 2 도메인 (log_size == 1)
if eval.domain.log_size() == 1 {
let y = eval.domain.half_coset.initial.y;
let n = BaseField::from(2);
let yn_inv = (y * n).inverse();
let y_inv = yn_inv * n;
let n_inv = yn_inv * y;
let (mut v0, mut v1) = (values[0], values[1]);
ibutterfly(&mut v0, &mut v1, y_inv);
return CirclePoly::new(vec![v0 * n_inv, v1 * n_inv]);
}


// 크기 4 도메인 (log_size == 2)
if eval.domain.log_size() == 2 {
let CirclePoint { x, y } = eval.domain.half_coset.initial;
let n = BaseField::from(4);
```

```
let xyn_inv = (x * y * n).inverse();
// ... 최적화된 4-point IFFT
}
```

Twiddle 사전 계산 최적화

```
fn slow_precompute_twiddles(mut coset: Coset) -> Vec<BaseField> {
let mut twiddles = Vec::with_capacity(coset.size());
for _ in 0..coset.log_size() {
let i0 = twiddles.len();
twiddles.extend(
coset
.iter()
.take(coset.size() / 2)
.map(|p| p.x) // x 좌표만 저장 (대칭성 활용)
.collect::<Vec<_>>(),
);
bit_reverse(&mut twiddles[i0..]); // 메모리 접근 최적화
coset = coset.double();
}
twiddles.push(1.into()); // 2의 거듭제곱 크기 맞춤
twiddles
```

```
}
```

적응적 알고리즘 선택

```
const MIN_FFT_LOG_SIZE: usize = 4; // 최소 SIMD FFT 크기
const CACHED_FFT_LOG_SIZE: usize = 12; // 캐시 최적화 임계값

// 크기에 따른 알고리즘 선택
if log_size < MIN_FFT_LOG_SIZE {
// CPU 백엔드로 폴백
let cpu_poly = eval.to_cpu().interpolate();
return CirclePoly::new(cpu_poly.coeffs.into_iter().collect());
}

if poly.log_size() <= 8 {
// 작은 다항식은 직접 평가
return slow_eval_at_point(poly, point);
}
```

2.7 성능 향상 효과

Circle FFT의 최적화 기법들은 다음과 같은 성능 향상을 제공합니다:

1. **SIMD 벡터화**: 16배 병렬 처리로 처리량 대폭 증가
2. **메모리 최적화**: 캐시 미스 감소로 실제 성능 향상

3. **병렬 처리**: 멀티코어 활용으로 확장성 개선

4. **적응적 알고리즘**: 크기별 최적 알고리즘 선택으로 전체 성능 최적화

5. **대칭성 활용**: Circle 기하학의 특성으로 연산량 감소

이러한 최적화들이 결합되어 Circle FFT는 기존 FFT 대비 실질적인 성능 향상을 달성하면서도 더 강한 보안성을 제공합니다.

결론

Circle STARK의 핵심인 Circle 기하학과 Circle FFT는 기존 STARK 시스템 대비 다음과 같은 혁신을 제공합니다:

1. **수학적 우아함**: 복소수 원의 기하학적 특성을 활용한 자연스러운 구조

2. **보안성 강화**: 더 복잡한 대수적 구조를 통한 공격 저항성 향상

3. **효율성**: Circle 특유의 대칭성을 활용한 최적화 기법들

4. **확장성**: 계층적 구조를 통한 다양한 크기 지원

이러한 특성들이 결합되어 Stwo가 StarkNet 생태계에서 차세대 영지식 증명 시스템으로 자리잡을 수 있는 기술적 기반을 제공합니다.